

Tài liệu QA Evidence Dependency Usage Analysis cho Migration Ant / TERASOLUNA

Mục tiêu: xác định dependency thật sự được sử dụng để giảm scope migration một cách có bằng chứng

Context: Dự án Java legacy đang dùng Apache Ant. Một số module như keilib và servercommon không có build.xml riêng, mà được build chung từ module KEIKA. KEIKA/build.xml trỏ tới các thư viện dùng chung như /lib/terasoluna5, /lib/test và /lib/extension. all.userlibraries có thể là cấu hình Eclipse, nhưng không nên xem là source of truth nếu chưa chứng minh được nó trùng với Ant build/runtime classpath.

Nội dung chính

1. Mục tiêu điều tra và nguyên tắc evidence
2. Tại sao không thể dựa hoàn toàn vào all.userlibraries
3. Năm lớp evidence cần thu thập
4. Cách phân loại dependency để giảm scope migration
5. Cấu trúc folder evidence đề xuất
6. Script mẫu để scan dependency theo module
7. Cách trình bày với QA/customer
8. Kết luận thực tế cho dự án hiện tại

1. Mục tiêu điều tra

Mục tiêu không phải là migrate toàn bộ JAR đang nằm trong thư mục TERASOLUNA, test hoặc extension. Mục tiêu đúng là tạo được bằng chứng rõ ràng để trả lời các câu hỏi sau:

- Dependency nào thật sự được project sử dụng?
- Dependency đó được sử dụng ở module nào: keilib, servercommon, KEIKA hay module khác?
- Bằng chứng sử dụng đến từ source code, compiled class, XML/JSP/properties, Ant classpath hay runtime log?
- Dependency đó là runtime dependency, test-only dependency, transitive dependency hay provided by application server?
- Dependency nào không có evidence và có thể đề xuất loại khỏi migration scope?

Nguyên tắc quan trọng: Không nói với QA/customer rằng một dependency chắc chắn không dùng chỉ vì không thấy import. Legacy Java có thể dùng dependency qua XML, JSP taglib, reflection, annotation, servlet container hoặc runtime configuration. Vì vậy cần kết hợp nhiều lớp evidence.

2. Không nên xem all.userlibraries là source of truth

Trong dự án hiện tại, all.userlibraries có vẻ chứa rất nhiều JAR tham chiếu tới các nhóm thư viện như TERASOLUNA, test libraries và extension libraries. Tuy nhiên file này thường phục vụ Eclipse IDE để resolve classpath trong workspace. Nó chỉ nên được xem là candidate dependency list, không phải source of truth chính thức nếu chưa có xác nhận.

Lý do all.userlibraries chưa đủ để kết luận:

- Nó có thể chứa nhiều thư viện cũ hoặc historical libraries không còn dùng.
- Nó có thể chứa source JAR, test JAR, server API JAR và runtime JAR lẫn lộn.
- Nó không chứng minh module nào đang dùng dependency nào.
- Nó không chứng minh Ant build thật sự đang dùng toàn bộ các JAR đó.
- Nó không chứng minh runtime server đang load toàn bộ các JAR đó.

Câu chốt: all.userlibraries cho biết Eclipse đang có sẵn những thư viện nào, nhưng không chứng minh toàn bộ module thật sự sử dụng tất cả thư viện đó.

3. Kết quả report QA cần tạo

Sau khi điều tra, nên tạo một dependency usage matrix để QA/customer review. Ví dụ:

Module	Dependency/JAR	Evidence	Evidence type	Action
keilib	spring-core	Import org.springframework.k trong source	Source import	Migrate
servercommon	jackson-databind	ObjectMapper trong JsonUtil.java	Source import	Migrate
KEIKA	mybatis / mybatis-spring	sqlMapConfig.xml hoặc mapper XML	Config/XML	Migrate
KEIKA	junit	Chỉ xuất hiện trong test source	Test only	Test scope
all	tomcat-servlet-api	Server cung cấp lúc runtime	Provided by server	Do not package
unknown	commons-compress	Không thấy source/jdeps/config/runtime evidence	No evidence	Candidate exclude

4. Năm lớp evidence cần thu thập

Không nên scan bằng một cách duy nhất. Với legacy Java/Spring/TERASOLUNA, dependency có thể được dùng ở nhiều nơi khác nhau. Nên dùng 5 lớp evidence sau:

```
Layer 1: Ant build classpath thật sự  
Layer 2: Source import scan theo từng module  
Layer 3: Bytecode scan bằng jdeps  
Layer 4: Config/JSP/XML/properties scan  
Layer 5: Runtime loaded-class evidence
```

Layer 1 - Lấy classpath thật sự từ Ant/KEIKA

Bước đầu tiên là chứng minh KEIKA build đang dùng classpath nào. Đây là bằng chứng quan trọng hơn all.userlibraries vì Ant build mới là thứ compile project thực tế.

Trong KEIKA/build.xml, tìm các đoạn tạo classpath dạng:

```
<path id="compile.classpath">  
  <fileset dir="${lib.dir}/terasoluna5">  
    <include name="*.jar"/>  
  </fileset>  
  <fileset dir="${lib.dir}/test">  
    <include name="*.jar"/>  
  </fileset>  
  <fileset dir="${lib.dir}/extension">  
    <include name="*.jar"/>  
  </fileset>  
</path>
```

Có thể thêm một target tạm vào build.xml để in classpath thật sự:

```
<target name="print-classpath">  
  <property name="resolved.classpath" refid="compile.classpath"/>  
  <echo message="=== COMPILE CLASSPATH ==="/>  
  <echo message="${resolved.classpath}"/>  
</target>
```

Sau đó chạy:

```
mkdir -p evidence  
ant print-classpath > evidence/01-ant-classpath.txt
```

Evidence tạo ra:

```
evidence/01-ant-classpath.txt
```

Thông điệp dùng cho QA:

```
This is the actual Ant build classpath used by KEIKA,  
not only Eclipse IDE configuration.
```

Layer 2 - Scan source import theo từng module

Mục tiêu là biết source code của keilib, servercommon và KEIKA đang import package nào.

```
mkdir -p evidence
```

```
find keilib servercommon KEIKA -name "*.java" -exec grep -H "^import " {} \; > evidence/02-source-imports.txt
```

Lọc danh sách import unique:

```
cat evidence/02-source-imports.txt | sed 's/.*import //' | sed 's/;///' | sort -u > evidence/02-source-imports-unique.txt
```

Ví dụ evidence có thể tìm được:

```
servercommon/src/main/java/.../JsonUtil.java:import com.fasterxml.jackson.databind.ObjectMapper;
keilib/src/main/java/.../DateUtil.java:import org.joda.time.DateTime;
KEIKA/src/main/java/.../UserMapper.java:import org.apache.ibatis.session.SqlSession;
```

Package pattern	Dependency group thường tương ứng
org.springframework.*	Spring Framework
org.springframework.security.*	Spring Security
com.fasterxml.jackson.*	Jackson
org.apache.ibatis.*	MyBatis
org.mybatis.spring.*	MyBatis Spring
org.terasoluna.gfw.*	TERASOLUNA GFW
org.joda.time.*	Joda-Time
org.slf4j.*	SLF4J
ch.qos.logback.*	Logback
org.apache.tiles.*	Apache Tiles
javax.servlet.*	Servlet API, thường là provided by server

Layer 3 - Dùng jdeps để scan compiled classes

Source import chưa đủ, vì code có thể dùng dependency qua annotation, inherited class, method signature hoặc generated class. Sau khi build xong, dùng jdeps để phân tích bytecode theo từng module.

```
mkdir -p evidence/jdeps
```

```
JARS=$(find ServerLib/lib/terasoluna5 ServerLib/lib/extension -name "*.jar" | tr ' ' ':')
```

```
jdeps -verbose:classpath -recursive -classpath "$JARS" keilib/build/classes > evidence/jdeps/keilib-jdeps.txt
```

```
jdeps -verbose:classpath -recursive -classpath "$JARS" servercommon/build/classes > evidence/jdeps/servercommon-jdeps.txt
```

```
jdeps -verbose:classpath -recursive -classpath "$JARS" KEIKA/build/classes > evidence/jdeps/KEIKA-jdeps.txt
```

Ví dụ evidence từ jdeps:

```
com.company.common.JsonUtil -> com.fasterxml.jackson.databind.ObjectMapper jackson-databind-2.13.1.jar
com.company.security.AuthUtil -> org.springframework.security.core.Authentication spring-security-core-5.4.2.jar
```

Ý nghĩa QA: jdeps là bằng chứng bytecode-level. Nếu jdeps cho thấy compiled class phụ thuộc vào package/class từ một JAR, dependency đó có evidence mạnh hơn việc chỉ nhìn vào danh sách JAR.

Layer 4 - Scan XML/JSP/properties/config

Legacy TERASOLUNA/Spring app thường dùng dependency trong XML, JSP, taglib hoặc properties. Những dependency này có thể không xuất hiện trong Java import.

```
find keilib servercommon KEIKA \(\ -name "*.xml" -o -name "*.properties" -o -name "*.jsp" -o -name
"*.tag" -o -name "*.tld" \) -exec grep -HnE "org\.springframework|org\.terasoluna|
org\.apache\.ibatis|com\.fasterxml\.jackson\.core|org\.apache\.tiles|org\.quartz|org\.dozer|joda|
logback|slf4j" {} \; > evidence/04-config-dependency-usage.txt
```

Ví dụ evidence từ config/JSP:

```
KEIKA/src/main/resources/spring/applicationContext.xml:
<bean class="org.mybatis.spring.SqlSessionFactoryBean">

KEIKA/src/main/webapp/WEB-INF/views/layout.jsp:
<%@ taglib prefix="sec" uri="http://www.springframework.org/security/tags" %>
```

Nhóm dependency thường phải scan kỹ trong XML/JSP:

- spring-webmvc
- spring-security-config
- spring-security-taglibs
- mybatis và mybatis-spring
- tiles-api, tiles-core, tiles-jsp
- taglibs-standard
- dozer / dozer-spring
- quartz
- logback hoặc slf4j binding

Layer 5 - Runtime loaded-class evidence

Đây là evidence mạnh khi QA/customer muốn biết lúc app chạy thật sự load JAR nào. Tuy nhiên runtime evidence phụ thuộc vào test coverage: nếu smoke test không chạy tới một flow nào đó, JAR liên quan có thể chưa được load.

Nếu dùng Java 8:

```
java -verbose:class ...
```

Nếu dùng Java 9 trở lên:

```
java -Xlog:class+load=info ...
```

Ví dụ capture log khi chạy app/test integration:

```
mkdir -p evidence/runtime

# Java 8 example
JAVA_OPTS="$JAVA_OPTS -verbose:class" ./run-server.sh > evidence/runtime/05-loaded-classes.log 2>&1
```

Extract danh sách JAR đã thật sự được load:

```
grep "\[Loaded" evidence/runtime/05-loaded-classes.log | grep ".jar" | sed -E 's/.*from //' |  
sort -u > evidence/runtime/05-runtime-loaded-jars.txt
```

Cách diễn đạt an toàn: Runtime loaded-class log chứng minh JAR đã được load trong phạm vi smoke/integration test đã chạy. Không nên dùng nó một mình để kết luận dependency không dùng ở toàn hệ thống nếu test coverage chưa đủ.

5. Cách phân loại dependency để giảm scope migration

Sau khi thu thập evidence, phân loại từng dependency thành các nhóm sau:

Nhóm	Ý nghĩa	Action đề xuất
A - Directly used by source	Có import trực tiếp trong Java source	Must migrate
B - Used by config/XML/JSP	Không thấy import nhưng xuất hiện trong XML, JSP, properties, taglib	Must migrate
C - Transitive dependency	Project không dùng trực tiếp nhưng dependency nhóm A/B cần nó	Keep as transitive, không declare thủ công nếu không cần
D - Test only	Chỉ dùng trong test source hoặc /lib/test	Test scope only
E - Provided by application server	Compile cần nhưng runtime server cung cấp	Provided scope / do not package
F - No evidence / unused candidate	Không thấy source, jdeps, config hoặc runtime evidence	Candidate to exclude, cần xác nhận rủi ro

Nhóm A - Directly used by source

Dependency có import trực tiếp trong Java source. Đây là nhóm có evidence rõ nhất.

```
com.fasterxml.jackson.databind.ObjectMapper  
org.springframework.stereotype.Service  
org.apache.ibatis.session.SqlSession
```

Action: Must migrate.

Nhóm B - Used by config/XML/JSP

Dependency không xuất hiện trong source import nhưng xuất hiện trong XML, JSP hoặc taglib.

```
spring-security-taglibs  
tiles-jsp  
mybatis-spring  
dozer-spring4
```

Action: Must migrate.

Nhóm C - Transitive dependency

Project không import trực tiếp nhưng dependency chính cần nó. Ví dụ Jackson databind cần jackson-core và jackson-annotations; Spring module này kéo Spring core/jcl.

```
jackson-core  
jackson-annotations  
spring-core  
spring-jcl
```

Action: Keep as transitive. Nếu dùng Ivy/Maven resolver thì không cần khai báo thủ công toàn bộ.

Nhóm D - Test only

Chỉ dùng trong test source hoặc thư mục /lib/test. Không nên trộn nhóm này với runtime migration nếu mục tiêu trước mắt là runtime app.

- JUnit
- Mockito
- DBUnit
- Cobertura
- Spring Test
- POI dùng cho test fixture nếu có
- XStream nếu chỉ phục vụ test

Action: Test scope only.

Nhóm E - Provided by application server

Một số API cần lúc compile nhưng app server có thể cung cấp lúc runtime. Nếu package nhăm vào WAR có thể gây conflict classloader.

```
servlet-api
jsp-api
el-api
tomcat-servlet-api
tomcat-jsp-api
tomcat-el-api
```

Action: provided scope / do not package into WAR, nhưng phải xác nhận app server version.

Nhóm F - No evidence / unused candidate

Không thấy evidence từ source import, jdeps, XML/JSP/properties hoặc runtime loaded-class log.

```
No evidence found in current static/runtime analysis.
Recommend excluding from migration unless confirmed by hidden runtime flow,
reflection, batch job, external plugin, or customer-specific operation.
```

Không nên nói tuyệt đối: Không nên viết “unused” nếu chưa test toàn bộ hệ thống. Nên viết “no evidence found / candidate for exclusion”.

6. Cấu trúc folder evidence đề xuất

```
migration-evidence/
├── 01-ant-classpath.txt
├── 02-source-imports.txt
├── 02-source-imports-unique.txt
├── 03-jar-package-index.txt
├── 04-config-dependency-usage.txt
├── jdeps/
│   ├── keilib-jdeps.txt
│   ├── servercommon-jdeps.txt
│   └── KEIKA-jdeps.txt
├── runtime/
│   ├── 05-loaded-classes.log
│   └── 05-runtime-loaded-jars.txt
└── report/
    └── dependency-usage-matrix.csv
```

7. Script tạo package index từ JAR

Mục tiêu là biết mỗi JAR chứa package/class nào để map import hoặc jdeps result ngược lại dependency/JAR.

```
mkdir -p evidence

for jar in $(find ServerLib/lib -name "*.jar" | sort); do
    echo "### $jar" >> evidence/03-jar-package-index.txt
    jar tf "$jar" | grep "\.class$" | sed 's|/|.|g' | sed 's|.class$||' | sort >>
evidence/03-jar-package-index.txt
done
```

Ví dụ dùng package index để map:

```
import com.fasterxml.jackson.databind.ObjectMapper

=> jackson-databind-2.13.1.jar contains com.fasterxml.jackson.databind.ObjectMapper
```

8. Script scan dependency theo module

Có thể tạo file scripts/scan-module-deps.sh:

```
#!/usr/bin/env bash
set -euo pipefail

MODULES=("keilib" "servercommon" "KEIKA")
EVIDENCE_DIR="evidence"

mkdir -p "$EVIDENCE_DIR/source" "$EVIDENCE_DIR/config" "$EVIDENCE_DIR/jdeps"

echo "Scanning source imports..."

for module in "${MODULES[@]}; do
    if [ -d "$module" ]; then
        find "$module" -name "*.java" -exec grep -H "^import " {} \; | sort -u >
"$EVIDENCE_DIR/source/${module}-imports.txt" || true

        find "$module" \( -name "*.xml" -o -name "*.properties" -o -name "*.jsp" -o -name "*.tag"
-o -name "*.tld" \) -exec grep -HnE "org\.springframework|org\.terasoluna|
org\.apache\.ibatis|com\.fasterxml|javax\.servlet|org\.apache\.tiles|org\.quartz|org\.dozer|joda|
logback|slf4j" {} \; > "$EVIDENCE_DIR/config/${module}-config-usage.txt" || true
    fi
done

echo "Done. Evidence generated under $EVIDENCE_DIR"
```

9. Cách nói với QA/customer

Nên trình bày theo hướng risk-based migration, không nói rằng dependency không dùng một cách tuyệt đối.

We should not assume that all JARs listed in `all.userlibraries` are required application dependencies.

The file appears to be an Eclipse IDE library configuration and includes runtime, test, extension, server API, source JAR references, and possibly historical libraries.

To reduce migration scope safely, we will perform dependency usage analysis using:

1. Actual Ant build classpath from KEIKA/build.xml
2. Java source import scan per module
3. Bytecode-level analysis using `jdeps`
4. XML/JSP/properties configuration scan
5. Runtime loaded-class evidence from smoke/integration tests

Based on the evidence, dependencies will be classified as:

- directly used
- config/runtime used
- transitive required
- test-only
- server-provided
- no evidence found / candidate for exclusion

Only dependencies with direct, config, bytecode, runtime, or transitive evidence should be included in the migration scope.

10. Đoạn ngắn để hỏi khách hàng hoặc senior

Hi team, to prepare QA evidence and reduce unnecessary dependency migration scope, I would like to perform a dependency usage analysis instead of assuming every JAR in `all.userlibraries` must be migrated.

Since modules like `keilib` and `servercommon` do not have their own build files and appear to be built through KEIKA, I will first capture the actual KEIKA Ant build classpath. Then I will scan source imports, compiled bytecode with `jdeps`, XML/JSP/properties configuration, and runtime loaded classes.

The goal is to classify each library as directly used, config/runtime used, transitive required, test-only, server-provided, or no evidence found. This will help us justify which dependencies must be migrated and which can be excluded or left out of scope.

Could you please confirm whether this approach is acceptable for QA evidence?

11. Kết luận thực tế cho case hiện tại

Với case hiện tại, không nên bắt đầu bằng việc khai báo Ivy dependency theo từng module ngay, vì `keilib` và `servercommon` không có `build.xml` riêng và có vẻ được build chung thông qua KEIKA.

Thứ tự nên làm:

9. Xem `all.userlibraries` là candidate list, không phải source of truth.
10. Extract actual KEIKA Ant classpath từ `build.xml`.
11. Scan source imports của `keilib`, `servercommon` và KEIKA.
12. Build project rồi chạy `jdeps` trên compiled output theo từng module.
13. Scan XML/JSP/properties để tìm dependency dùng qua config.
14. Capture runtime loaded JARs từ smoke/integration test nếu có thể.
15. Tạo dependency usage matrix.
16. Chỉ migrate dependency có evidence hoặc là transitive dependency bắt buộc.

Câu chốt cho QA: all.userlibraries shows what is available to Eclipse, but not necessarily what each module actually uses. The migration scope should be based on actual Ant build classpath plus source, bytecode, configuration, and runtime evidence.

12. Gợi ý format dependency usage matrix CSV

```
Module,Dependency,Jar,Version,Evidence Type,Evidence File,Evidence  
Detail,Classification,Action,Risk Note  
servercommon,Jackson,jackson-databind,2.13.1,Source import,evidence/source/servercommon-  
imports.txt,JsonUtil imports ObjectMapper,Directly used,Migrate,  
KEIKA,MyBatis,mybatis-spring,2.0.6,Config XML,evidence/config/KEIKA-config-  
usage.txt,SqlSessionFactoryBean in applicationContext.xml,Config used,Migrate,  
KEIKA,JUnit,junit,4.12,Test source,evidence/source/KEIKA-test-imports.txt,Only used under test,Test  
only,Test scope only,  
ALL,Servlet API,tomcat-servlet-api,9.0.46,Provided server,server confirmation,Provided by  
Tomcat,Provided,Do not package,Confirm server version  
UNKNOWN,commons-compress,commons-compress,1.20,No evidence,evidence summary,No  
source/jdeps/config/runtime evidence,No evidence,Candidate exclude,Need confirm no hidden flow
```

13. Final recommendation

Đề xuất chính thức: sử dụng phương pháp evidence-based dependency reduction. Không migrate toàn bộ thư viện chỉ vì chúng tồn tại trong all.userlibraries hoặc thư mục /lib. Trước tiên cần chứng minh dependency thật sự được dùng thông qua Ant classpath, source import, bytecode jdeps, config scan và runtime evidence. Sau đó mới quyết định dependency nào phải migrate, dependency nào chỉ test scope, dependency nào do server provide, và dependency nào có thể loại khỏi scope migration.